ChatGPT

CA TUCR HTLA Inverse Bridge Formalization

Architectural reading of the inverse bridge

The forward bridge you already ratified takes three continuous scalars — E_{err}, RPE, and Novelty — and feeds them into the bounded 11D ODE, which then drives the sparsity gate that performs the k-WTA wipeout on the 1024D quinary substrate. The same bridge document also makes the substrate constraints explicit: the lattice is quinary, the gate is bounded, and the masking rule must preserve the vacuum invariant $P_A = 0 \Rightarrow P_S = 0$. filecite turn0file0

For the inverse bridge, the cleanest way to close the loop is not to force a Euclidean geometry onto a sparse quinary lattice, but to use the same families of similarity measures that the HDC/VSA literature already treats as natural for discrete hypervectors: Hamming-style agreement for discrete/binary structure, and cosine or normalized dot-product style similarity for integer-valued hypervectors. The literature is explicit that the "right" similarity is representation-dependent and application-specific; for integer hypervectors, normalized dot products preserve intensity or frequency information that plain cosine can suppress. [1]

That separation also lines up with the neuroscience distinction your continuous layer already assumes. General prediction error is a mismatch signal. Reward prediction error is a signed valuation of that mismatch, canonically "actual minus expected reward." Novelty is not the same thing as either of those; in the anomaly-detection literature it is typically modeled as distance from the learned support of normal data, often with nearest-neighbor or residual-to-subspace constructions. [2]

What follows is therefore a **ratified composite formalism**: a weighted quinary bit-plane mismatch for E_{err}, a value-axis projection difference for RPE, and a local-manifold residual plus neighborhood-distance score for Novelty. The exact package is architecture-specific, but each ingredient is directly grounded in standard similarity, prediction-error, and novelty-detection machinery. [3]

Ratified reduction for general prediction error

The right starting point is a **vacuum-safe quinary encoding**. For each lattice coordinate $x_i \in \{-2, -1, 0, 1, 2\}$, define three HTLA-compatible planes:

$$a_i(x) = \mathbf{1}[x_i \neq 0] \quad =$$

$$s_i(x) = \mathbf{1}[x_i < 0]$$

$$u_i(x) = \mathbf{1}[|x_i| = 2]$$

This gives the exact code table

$$0 \mapsto (0,0,0), \quad +1 \mapsto (1,0,0), \quad -1 \mapsto (1,1,0), \quad +2 \mapsto (1,0,1), \quad -2 \mapsto (1,1,1)$$

which automatically enforces the vacuum invariant because $a_i = 0$ forces the sign and strength planes to be irrelevant. That is the key reason to compare vectors in structured plane-space rather than with Euclidean distance. The HDC/VSA literature treats precisely these kinds of discrete agreement measures as the natural similarity machinery for high-dimensional symbolic vectors. [4]

For expected and actual hypervectors $H_{\text{exp}}, H_{\backslash\text{act}}$, define the per-coordinate mismatch cost

$$c_i \;=\; \alpha\,|a_i^{\text{exp}} - a_i^{\backslash\text{act}}| \;+\; \alpha_2\,|a_i^{\text{exp}} - a_i^{\backslash\text{act}}|\,\max(u_i^{\text{exp}}, u_i^{\backslash\text{act}}) \;+\; \beta\,a_i^{\text{exp}} a_i^{\backslash\text{act}} |s_i^{\text{exp}} - s_i^{\backslash\text{act}}| \;+\; \gamma\,a_i^{\text{exp}} a_i^{\backslash\text{act}} |u_i^{\text{exp}} - u_i^{\backslash\text{act}}|$$

with the **ratified weights**

$$\alpha = \frac{1}{4}, \qquad \alpha_2 = \frac{1}{8}, \qquad \beta = \frac{1}{2}, \qquad \gamma = \frac{1}{8}$$

and

$$c_{\max} = \max(\alpha + \alpha_2,\ \beta + \gamma) = \frac{5}{8}.$$

Then define the effective support size

$$n_{\cup} = \max\left(1, \sum_{i=1}^{1024} \max(a_i^{\text{exp}}, a_i^{\backslash\text{act}})\right)$$

and the scalar prediction error

$$E_{\text{err}} = \frac{1}{c_{\max} n_{\cup}} \sum_{i=1}^{1024} c_i.$$

This is exactly equivalent to a weighted popcount/Hamming construction on the active, sign, and high-magnitude planes:

$$E_{\text{err}} = \frac{\alpha\ \text{popcnt}(A_{\text{exp}} \oplus A_{\backslash\text{act}}) + \alpha_2\ \text{popcnt}\big((A_{\text{exp}} \oplus A_{\backslash\text{act}}) \wedge (U_{\text{exp}} \vee U_{\backslash\text{act}})\big) + \beta\ \text{popcnt}\big((A_{\text{exp}} \wedge A_{\backslash\text{act}}) \wedge (S_{\text{exp}} \oplus S_{\backslash}$$

The important behavior is baked in:

$$0 \leftrightarrow \pm 1 \;\mapsto\; 0.4, \qquad 0 \leftrightarrow \pm 2 \;\mapsto\; 0.6, \qquad +1 \leftrightarrow -1 \;\mapsto\; 0.8, \qquad +1 \leftrightarrow +2 \;\mapsto\; 0.2, \qquad +1 \leftrightarrow -2 \;\mapsto\; 1.0.$$

So **vacuum mismatch**, **sign contradiction**, and **magnitude error** are all distinct, and direct contradiction is penalized more strongly than a simple occupancy mismatch, exactly as required. Joint vacuums do **not** artificially inflate similarity, because the denominator is the active union $n_{\cup}$, not the full 1024 dimensions. That is the correct normalizer for a sparse quinary substrate. [4]

Ratified reduction for reward prediction error

Your own bridge document already treats E_{err} as value-neutral and RPE as a different scalar channel, with distinct continuous-layer consequences. That distinction is also consistent with the neuroscience literature:

reward prediction error is a signed discrepancy between actual and expected value, whereas prediction-error signaling more broadly can also track value-neutral feature violations. filecite turn0file0 [5]

To build a value scalar from quinary HTLA states, first collapse the three planes back into a **signed strength embedding**:

$$q_i(H) = \big(1 + u_i(H)\big)\, a_i(H)\, \big(1 - 2s_i(H)\big) \in \{-2, -1, 0, 1, 2\}.$$

This is just the original quinary symbol recovered in a plane-safe way. Then define the **value-axis projection** onto the immutable Value Anchor $H_{\setminus \mathrm{val}}$ by

$$V(H \mid H_{\setminus \mathrm{val}}) = \frac{\langle q(H), q(H_{\setminus \mathrm{val}}) \rangle}{2\|q(H_{\setminus \mathrm{val}})\|_1 + \varepsilon}.$$

This is a bounded **normalized dot projection**, not a cosine. That choice is deliberate. In HDC/VSA settings, normalized dot products preserve intensity information that cosine suppresses; here, that matters because ± 2 must remain more value-salient than ± 1. [6]

The signed reward prediction error is then

$$\mathrm{RPE} = \mathrm{clip}\left(\frac{V(H_{\setminus \mathrm{act}} \mid H_{\setminus \mathrm{val}}) - V(H_{\mathrm{exp}} \mid H_{\setminus \mathrm{val}})}{2}, -1, 1 \right).$$

Interpretation is straightforward:

- $V(H_{\setminus \mathrm{act}} \mid H_{\setminus \mathrm{val}}) > V(H_{\mathrm{exp}} \mid H_{\setminus \mathrm{val}})$ gives positive RPE, meaning reality landed **better** than expected along the value axis.
- $V(H_{\setminus \mathrm{act}} \mid H_{\setminus \mathrm{val}}) < V(H_{\mathrm{exp}} \mid H_{\setminus \mathrm{val}})$ gives negative RPE, meaning reality landed **worse** than expected.
- Large neutral mismatches can still produce near-zero RPE if they are orthogonal to the value anchor. That is desirable, because neutral structural error belongs in E_{err}, not in RPE. [7]

If you prefer a cosine-style variant, the quinary-safe substitute is

$$V_{\cos}(H \mid H_{\setminus \mathrm{val}}) = \frac{\langle q(H), q(H_{\setminus \mathrm{val}}) \rangle}{\|q(H)\|_2 \|q(H_{\setminus \mathrm{val}})\|_2 + \varepsilon},$$

but I would **not** ratify it as primary because it collapses ± 1 and ± 2 when they are collinear with the anchor. For this architecture, retaining quinary intensity is the better fit. [6]

Ratified reduction for novelty

Novelty should not be reduced to “distance from expectation” alone. In the novelty-detection literature, the usual problem is one-class support estimation: identify whether a point sits inside the learned support of the “warm” class or outside it. Standard tools include nearest-neighbor distance, local density methods such as LOF, and residual distance to a learned subspace or manifold. [8]

Let the warm memory manifold be represented by a memory bank

$$\mathcal{W} = \{M_1, \dots, M_N\}, \qquad M_j \in \{-2, -1, 0, 1, 2\}^{1024}.$$

First compute quinary mismatch distances to all warm memories using the same E_{err} metric:

$$D_j = E_{\text{err}}(M_j, H_{\setminus \text{act}}).$$

Take the K nearest warm neighbors

$$\mathcal{N}_K(H_{\setminus \text{act}}) = \{M_{(1)}, \dots, M_{(K)}\}.$$

From those, define the **local density deficit**

$$\rho_K(H_{\setminus \text{act}}) = \frac{1}{K} \sum_{M \in \mathcal{N}_K(H_{\setminus \text{act}})} E_{\text{err}}(M, H_{\setminus \text{act}}),$$

which already lives in $[0, 1]$.

Now approximate the local manifold with a rank-r linear subspace in the signed quinary embedding $q(\cdot)$. Let

$$\mu = \frac{1}{K} \sum_{M \in \mathcal{N}_K(H_{\setminus \text{act}})} q(M)$$

and let $U_r \in \mathbb{R}^{1024 \times r}$ contain the top r right-singular vectors of the centered neighbor matrix. Then the **orthogonal residual energy ratio** is

$$\eta_\perp(H_{\setminus \text{act}}) = \frac{\left\| \left(I - U_r U_r^\top\right) \left(q(H_{\setminus \text{act}}) - \mu\right) \right\|_2^2}{\|q(H_{\setminus \text{act}}) - \mu\|_2^2 + \varepsilon}.$$

This is the normalized orthogonal distance from the local warm manifold; the subspace/residual interpretation is standard in PCA-style monitoring and class-subspace methods. [9]

The ratified novelty scalar is then the convex blend

$$\text{Novelty} = \lambda\, \eta_\perp(H_{\setminus \text{act}}) + (1 - \lambda)\, \rho_K(H_{\setminus \text{act}}), \qquad \lambda = \frac{1}{2}.$$

That choice does two useful things at once:

- ρ_K says whether the state is locally **far from what is warm**.
- $\eta_\perp$ says whether the state is **off-manifold**, even if some neighbors are nearby.

That is closer to your stated requirement than either kNN distance alone or PCA residual alone. If the full HSW graph is available at runtime, $\mathcal{W}$ should be the current warm neighborhood. If it is not, the safe operational fallback is

$$\mathcal{W} = \{H_{\text{exp}}\},$$

which reduces novelty to a degenerate one-point manifold without breaking the API.

Why the outputs are safely bounded

For E_{err}, every coordinate cost satisfies $0 \le c_i \le c_{\max}$, and nonzero costs can only occur on the active union support. Therefore

$$0 \le \sum_i c_i \le c_{\max} n_{\cup},$$

so

$$0 \le E_{\text{err}} \le 1.$$

The denominator never vanishes because $n_{\cup}$ is replaced by $\max(1, n_{\cup})$. If both vectors are entirely vacuum, every $c_i = 0$, so $E_{\text{err}} = 0$. No undefined branch appears. filecite turn0file0

For the value projection, each coordinate of $q(H)$ lies in $[-2, 2]$, so

$$|\langle q(H), q(H_{\setminus \text{val}})\rangle| \le \sum_i |q_i(H)|\,|q_i(H_{\setminus \text{val}})| \le 2 \sum_i |q_i(H_{\setminus \text{val}})| = 2\|q(H_{\setminus \text{val}})\|_1.$$

Hence

$$-1 \le V(H \mid H_{\setminus \text{val}}) \le 1.$$

It follows immediately that

$$-1 \le \frac{V(H_{\setminus \text{act}} \mid H_{\setminus \text{val}}) - V(H_{\text{exp}} \mid H_{\setminus \text{val}})}{2} \le 1,$$

so RPE is safely bounded in $[-1, 1]$. If $H_{\setminus \text{val}}$ is all vacuum, the denominator is zero in the mathematical idealization; the implementation returns 0, which is the only neutral choice that does not inject fictitious value. That is also consistent with the HDC/VSA literature's "application-specific metric" guidance: once the anchor is null, the value axis is undefined, so the signed value difference should collapse to zero. [10]

For Novelty, the neighbor distance term satisfies $0 \le \rho_K \le 1$ because it is an average of E_{err} scores. The orthogonal residual satisfies $0 \le \eta_{\perp} \le 1$ because it is a residual-energy fraction. A convex combination of two values in $[0, 1]$ remains in $[0, 1]$, so Novelty is bounded. If there are too few neighbors to fit a local subspace, the implementation falls back to ρ_K; if memory is entirely absent, the safe convention is Novelty $= 0$ for the all-vacuum state and Novelty $= 1$ for any non-vacuum state. Those fallback rules are exactly the kinds of support-based edge cases the novelty-detection literature treats as necessary in one-class settings. [11]

NumPy reference implementation

```
import numpy as np

# Ratified plane weights for E_err
ALPHA_OCC = 0.25       # vacuum/non-vacuum mismatch
ALPHA_OCC_U2 = 0.125   # extra surcharge if the missing activity is |2|
BETA_SIGN = 0.50       # direct sign contradiction
GAMMA_MAG = 0.125      # |1| vs |2| mismatch when both active
C_MAX = max(ALPHA_OCC + ALPHA_OCC_U2, BETA_SIGN + GAMMA_MAG)  # = 0.625
EPS = 1e-12


def _validate_quinary(x: np.ndarray, name: str = "vector") -> np.ndarray:
    x = np.asarray(x, dtype=np.int8).reshape(-1)
    if x.ndim != 1:
        raise ValueError(f"{name} must be a 1D array.")
    if not np.all(np.isin(x, np.array([-2, -1, 0, 1, 2], dtype=np.int8))):
        raise ValueError(f"{name} contains values outside {{-2,-1,0,1,2}}.")
    return x


def _planes(h: np.ndarray):
    """
    HTLA-compatible quinary plane encoding.

    A: activity plane, 1 iff h != 0
    S: sign plane,     1 iff h < 0, otherwise 0
    U: unit-2 plane,   1 iff |h| == 2, otherwise 0

    Recover signed quinary symbol via:
        q = (1 + U) * A * (1 - 2*S)  in {-2,-1,0,1,2}
    """
    h = _validate_quinary(h)
    A = (h != 0).astype(np.uint8)
    S = (h < 0).astype(np.uint8)       # safe because A gates all sign usage
    U = (np.abs(h) == 2).astype(np.uint8)
    q = ((1 + U.astype(np.int8)) * A.astype(np.int8) * (1 - 2 *
S.astype(np.int8))).astype(np.int8)
    return A, S, U, q


def prediction_error(H_exp: np.ndarray, H_act: np.ndarray) -> float:
    """
    Weighted bit-plane Hamming/popcount mismatch on the active support union.
```

```
    Returns:
        E_err in [0, 1]
    """
    Ae, Se, Ue, _ = _planes(H_exp)
    Aa, Sa, Ua, _ = _planes(H_act)

    occ_mismatch = np.bitwise_xor(Ae, Aa).astype(np.float64)
    jointly_active = np.bitwise_and(Ae, Aa).astype(np.float64)
    sign_mismatch = jointly_active * np.bitwise_xor(Se, Sa).astype(np.float64)
    mag_mismatch = jointly_active * np.bitwise_xor(Ue, Ua).astype(np.float64)
    missing_u2 = occ_mismatch * np.bitwise_or(Ue, Ua).astype(np.float64)

    cost = (
        ALPHA_OCC * occ_mismatch
        + ALPHA_OCC_U2 * missing_u2
        + BETA_SIGN * sign_mismatch
        + GAMMA_MAG * mag_mismatch
    )

    support_union = int(np.bitwise_or(Ae, Aa).sum())
    denom = C_MAX * max(1, support_union)

    return float(np.clip(cost.sum() / denom, 0.0, 1.0))


def value_projection(H: np.ndarray, H_val: np.ndarray) -> float:
    """
    Bounded value-axis projection onto the immutable Value Anchor.

    Uses a normalized dot projection rather than cosine so that ±2 retains
    greater salience than ±1.

    Returns:
        V(H | H_val) in [-1, 1]
    """
    _, _, _, q = _planes(H)
    _, _, _, qv = _planes(H_val)

    q = q.astype(np.float64)
    qv = qv.astype(np.float64)

    denom = 2.0 * np.abs(qv).sum()
    if denom < EPS:
        return 0.0

    v = np.dot(q, qv) / denom
    return float(np.clip(v, -1.0, 1.0))
```

```
def reward_prediction_error(H_exp: np.ndarray, H_act: np.ndarray, H_val:
np.ndarray) -> float:
    """
    Signed reward prediction error:
        RPE = 0.5 * (V_act - V_exp)

    Returns:
        RPE in [-1, 1]
    """
    v_exp = value_projection(H_exp, H_val)
    v_act = value_projection(H_act, H_val)
    rpe = 0.5 * (v_act - v_exp)
    return float(np.clip(rpe, -1.0, 1.0))


def novelty_score(
    H_act: np.ndarray,
    warm_memory: np.ndarray | None = None,
    H_exp_fallback: np.ndarray | None = None,
    k: int = 8,
    subspace_rank: int = 4,
    manifold_weight: float = 0.5,
) -> float:
    """
    Novelty = convex combination of:
      - rho_K: mean prediction-error distance to K nearest warm memories
      - eta_perp: orthogonal residual energy to the local warm-memory manifold

    If warm_memory is None, fall back to the degenerate manifold {H_exp}.
    """
    H_act = _validate_quinary(H_act, "H_act")

    if warm_memory is None:
        if H_exp_fallback is None:
            warm_memory = H_act.reshape(1, -1)
        else:
            warm_memory = _validate_quinary(H_exp_fallback,
"H_exp_fallback").reshape(1, -1)

    W = np.asarray(warm_memory, dtype=np.int8)
    if W.ndim == 1:
        W = W.reshape(1, -1)

    if W.shape[0] == 0:
        return 0.0 if np.all(H_act == 0) else 1.0

    # Local density term using the same quinary prediction-error metric.
```

```
    dists = np.array([prediction_error(mem, H_act) for mem in W],
dtype=np.float64)
    k_eff = max(1, min(int(k), W.shape[0]))
    nn_idx = np.argsort(dists)[:k_eff]
    N = W[nn_idx]
    rho_k = float(np.clip(dists[nn_idx].mean(), 0.0, 1.0))

    # Local manifold term: orthogonal residual to PCA subspace of nearest
neighbors.
    X = N.astype(np.float64)
    x = H_act.astype(np.float64)
    mu = X.mean(axis=0)
    Z = X - mu

    if N.shape[0] < 2 or np.allclose(Z, 0.0):
        eta_perp = rho_k
    else:
        _, _, VT = np.linalg.svd(Z, full_matrices=False)
        r = min(int(subspace_rank), VT.shape[0], N.shape[0] - 1)

        if r <= 0:
            eta_perp = rho_k
        else:
            basis = VT[:r].T                    # shape: (D, r)
            z = x - mu
            denom = float(np.dot(z, z))

            if denom < EPS:
                eta_perp = 0.0
            else:
                z_parallel = basis @ (basis.T @ z)
                z_res = z - z_parallel
                eta_perp = float(np.clip(np.dot(z_res, z_res) / denom, 0.0,
1.0))

    novelty = manifold_weight * eta_perp + (1.0 - manifold_weight) * rho_k
    return float(np.clip(novelty, 0.0, 1.0))


def inverse_bridge_scalars(
    H_exp: np.ndarray,
    H_act: np.ndarray,
    H_val: np.ndarray,
    warm_memory: np.ndarray | None = None,
    k: int = 8,
    subspace_rank: int = 4,
    manifold_weight: float = 0.5,
) -> tuple[float, float, float]:
```

```
    """
    Required interface:
        takes Expected, Actual, and Value Anchor
        returns (E_err, RPE, Novelty)

    Optional:
        warm_memory can be supplied for the full manifold-aware novelty score.
        If omitted, novelty falls back to the degenerate manifold {H_exp}.
    """
    H_exp = _validate_quinary(H_exp, "H_exp")
    H_act = _validate_quinary(H_act, "H_act")
    H_val = _validate_quinary(H_val, "H_val")

    if H_exp.shape != H_act.shape or H_exp.shape != H_val.shape:
        raise ValueError("H_exp, H_act, and H_val must have identical shapes.")
    if H_exp.shape[0] != 1024:
        raise ValueError("All hypervectors must be 1024D.")

    E_err = prediction_error(H_exp, H_act)
    RPE = reward_prediction_error(H_exp, H_act, H_val)
    Novelty = novelty_score(
        H_act,
        warm_memory=warm_memory,
        H_exp_fallback=H_exp,
        k=k,
        subspace_rank=subspace_rank,
        manifold_weight=manifold_weight,
    )

    return E_err, RPE, Novelty
```

This implementation closes the discrete-to-continuous loop in a way that matches the forward bridge's requirements on bounded scalar drives, preserves the vacuum invariant by construction, and avoids undefined behavior in all of the edge cases that matter operationally: all-vacuum inputs, null value anchors, sparse warm-memory neighborhoods, and no-manifold fallbacks. filecite turn0file0 [3]

Final ratification

The mathematically safest inverse bridge for the CA/TUCR/HTLA stack is:

$$\boxed{E_{\text{err}} = \text{weighted active/sign/strength popcount mismatch on the active union}}$$

$$\boxed{\text{RPE} = \frac{1}{2}\Big(V(H_{\setminus \text{act}} \mid H_{\setminus \text{val}}) - V(H_{\text{exp}} \mid H_{\setminus \text{val}})\Big)}$$

$$\boxed{\text{Novelty} = \frac{1}{2}\,\eta_{\perp}(H_{\setminus \text{act}}) + \frac{1}{2}\,\rho_K(H_{\setminus \text{act}})}$$

with the quinary-safe plane encoding

$$q_i = (1 + u_i)a_i(1 - 2s_i).$$

That package is the best fit for your stated constraints because it does all three things simultaneously: it respects the quinary vacuum semantics, distinguishes absence mismatches from signed contradictions, and maps cleanly into the exact bounded scalar intervals the 11D ODE already expects. filecite turn0file0

---

1 3 4 6 10 Hyperdimensional computing: a framework for stochastic computation and symbolic AI | Journal of Big Data | Springer Nature Link
https://link.springer.com/article/10.1186/s40537-024-01010-8?utm_source=chatgpt.com

2 5 7 Dopamine reward prediction errors reflect hidden-state inference across time - PubMed
https://pubmed.ncbi.nlm.nih.gov/28263301/?utm_source=chatgpt.com

8 11 A review of novelty detection - ORA - Oxford University Research Archive
https://ora.ox.ac.uk/objects/uuid%3A947eb5ef-ea04-4156-840a-ae957d35d4f6?utm_source=chatgpt.com

9 On the orthogonal distance to class subspaces for high-dimensional data classification - ScienceDirect
https://www.sciencedirect.com/science/article/abs/pii/S0020025517308356?utm_source=chatgpt.com